

# Vulnerable App Pipeline

## Risk controls summary and evidence (for FDA)

Sat Jun 13 2026 19:52:50 GMT+0000 (Coordinated Universal Time)

**Description:** *No description*  
**Owner:** Tresor Matondo  
**Inherent Risk Level:** High (73%)  
**Current Risk Level:** High (73%)

## Content menu

### FDA Summary Information

- [Threat Modeling and Risk Assessment](#)
- [Risk Distribution](#)
- [Controls - Countermeasures state](#)

### Security risk Management

- [Security risk management plan](#)

### Security Architect Views

- [The components of the system architecture](#)
  - [Global system view](#)
- [The cybersecurity threats to system components](#)
  - [Component details](#)
- [Countermeasures to address those threats](#)
  - [Multi-Patient harm view](#)
  - [Updateability / Patchability view](#)
  - [Security Use Case View](#)
- [Countermeasures validation or testing](#)
  - [Cybersecurity risk controls summary](#)

## FDA Summary information

This section summarizes the cybersecurity status of the device based on the threat modeling analysis, mitigation coverage, and testing results. All information comes directly from the IriusRisk threat model and related countermeasure validation.

### Threat Modeling overview

Number of threats identified and the current status of mitigations.

Total Threats identified: 17

---

### Threat mitigation coverage

Fully mitigated: 0

Partly mitigated: 0

Not yet mitigated: 17

Residual high-severity remaining: 0

*Residual Risk refers to the risk that remains after implementing controls or when a deliberate decision has been made to accept the risk.*

---

### Threat distribution by severity

Breakdown of hazards by potential impact and severity:

Very Low:

Low:

Medium:

High:

Critical:

0

0

3

5

9

Note: Severity ratings are qualitative, derived from IriusRisk's scoring methodology.

### Controls and testing

The current status and priority of all security countermeasures.

Total Countermeasures identified: 18

---

### Countermeasure status

Implemented: 0

Required: 0

Recommended: 18

Rejected: 0

N/A: 0

---

### Countermeasures testing results

Test with Error: 0

Test Passed: 0

[Risk Calculation Algorithm](#)

## Security risk management

Threat Modeling was performed using the IriusRisk scoring methodology, following the approach defined in the Cybersecurity Threat Modeling framework.

### Security risk management plan

The IriusRisk platform was used to:

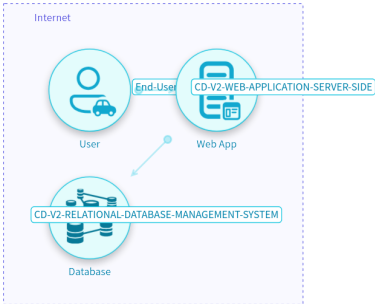
- Model the system architecture.
- Map threats to mitigations and test results.
- Track residual risk acceptance.

## Security Architect Views

This section provides end-to-end views of the project, connectivity, and defensive design patterns, demonstrating how risks have been identified and mitigated at every level. It consolidates threat modeling results, security architecture views, and cybersecurity risk controls in compliance with FDA, demonstrating how risks have been identified and mitigated at every level.

### The components of the system architecture

#### Global system view



#### System Components:

- Database
- User
- Web App

The cybersecurity threats to system components

Component details

Component: Database

Threats of this component

T-1 An attacker can exploit SQL injection vulnerabilities

SQL injection is a type of security vulnerability that occurs when an attacker can manipulate or inject malicious SQL code into queries executed by an application, typically through user inputs. In the context of an RDBMS, this can lead to unauthorized access, data manipulation, and potentially full compromise of the database.

Threat Agents/Attack Vectors:

- **Malicious Input:** Attackers can exploit SQL injection by injecting malicious SQL statements or commands into user inputs such as form fields, URL parameters, or cookies.
- **Tainted Data:** If user input is not properly validated or sanitized before being included in SQL queries, an attacker can manipulate the input to inject malicious SQL code.
- **Stored Procedures:** In cases where stored procedures are used, attackers may exploit vulnerabilities by manipulating the input parameters or injecting malicious code directly into the stored procedures.

Impacts:

- **Unauthorized Access:** SQL injection can allow attackers to gain unauthorized access to sensitive data in the database.
- **Data Manipulation:** Attackers can manipulate or delete data within the database, leading to data integrity issues.
- **Code Execution:** In some cases, successful SQL injection can lead to the execution of arbitrary code on the database server.

Example Attack Scenarios:

Authentication Bypass:

- *Attack Vector:* Injecting malicious SQL code into a login form.
- *Impact:* Gain unauthorized access to the application by bypassing authentication.

Data Exfiltration:

- *Attack Vector:* Injecting SQL code to extract sensitive information.
- *Impact:* Retrieve and exfiltrate confidential data from the database.

Database Defacement:

- *Attack Vector:* Injecting SQL code to modify data in the database.
- *Impact:* Modify or delete records, leading to data manipulation or defacement.

Secondary Exploitation:

- *Attack Vector:* Injecting SQL code to execute additional malicious queries.
- *Impact:* Exploit further vulnerabilities, potentially leading to full compromise of the database server.

• Risk level: Critical      • State: Risk exposed

T-2 An attacker can gain unauthorized access to the database

Unauthorized access to a database refers to the threat of an attacker gaining entry into a database without the proper permissions or authentication. This could result in the compromise of sensitive data, data manipulation, or potential misuse of the database resources.

Threat Agents/Attack Vectors:

- **Brute Force Attacks:** Attackers may attempt to gain unauthorized access by systematically trying various username and password combinations until the correct credentials are found.
- **Exploiting Weak Authentication:** Weak or default credentials are a common target. Attackers may exploit weak passwords or default login credentials to gain access.
- **SQL Injection:** If the application is vulnerable to SQL injection, attackers might exploit this weakness to bypass authentication mechanisms and gain unauthorized access.
- **Credential Theft:** Phishing attacks or malware on the server hosting the database could lead to the theft of authentication credentials.

Impacts:

- **Data Exposure:** Unauthorized access could lead to the exposure of sensitive or confidential information stored in the database.
- **Data Manipulation:** Attackers may modify or delete data, impacting the integrity of the information stored.
- **Service Disruption:** Unauthorized access may allow attackers to disrupt database services, causing denial of service or other operational issues.

Example Attack Scenarios:

Brute Force Attack:

- *Attack Vector:* Repeatedly attempting different username and password combinations.
- *Impact:* Gain access to the database with valid credentials through trial and error.

Default Credentials Exploitation:

- *Attack Vector:* Exploiting databases using default or weak credentials.
- *Impact:* Unauthorized access to the database due to insecure default configurations.

SQL Injection for Authentication Bypass:

- *Attack Vector:* Injecting SQL code to manipulate authentication checks.
- *Impact:* Bypass authentication mechanisms and gain unauthorized access.

Credential Theft:

- *Attack Vector:* Phishing attack targeting database administrators.
- *Impact:* Obtaining valid credentials for unauthorized access.

• Risk level: Critical      • State: Risk exposed

**T-3 An attacker can perform data exfiltration through insecure backups**

Data exfiltration through insecure backups refers to the threat of an attacker exploiting vulnerabilities or misconfigurations in the backup process to gain unauthorized access to and extract sensitive data from backup files. This can lead to the unauthorized disclosure of sensitive information stored in those backups.

**Threat Agents/Attack Vectors:**

- **Unauthorized Access to Backup Storage:** Exploiting weak or default credentials, misconfigured access controls, or vulnerabilities in the backup storage system to gain access to backup files.
- **Manipulating Backup Processes:** Tampering with the backup processes to include malicious files or to modify the backup content.
- **Interception of Backup Data in Transit:** Intercepting or eavesdropping on the communication channels used to transfer backup data between systems.

**Impacts:**

- **Unauthorized Data Access:** The attacker gains access to sensitive data contained in backup files.
- **Data Exposure:** Sensitive information stored in backups is exposed to unauthorized parties.
- **Data Manipulation:** Attackers may modify or inject malicious data into backup files.

**Example Attack Scenarios:**

**Exploiting Weak Backup Storage Security:**

- *Attack Vector:* Gaining unauthorized access to backup storage due to weak credentials or misconfigurations.
- *Impact:* Extracting sensitive data from backup files stored in the compromised backup system.

**Manipulating Backup Processes:**

- *Attack Vector:* Tampering with backup processes to include malicious files or modify backup content.
- *Impact:* Unauthorized access and extraction of manipulated or malicious data during the restoration process.

**Interception of Backup Data in Transit:**

- *Attack Vector:* Intercepting communication channels used to transfer backup data.
- *Impact:* Capturing sensitive data in transit, leading to unauthorized access and potential data exfiltration.

**Abusing Backup Encryption Weakness:**

- *Attack Vector:* Exploiting weaknesses in backup encryption mechanisms.
- *Impact:* Decrypting or bypassing backup encryption to gain access to sensitive information.

- **Risk level:**  Critical
- **State:**  Risk exposed

**T-4 An attacker can perform database privilege escalation**

Database privilege escalation involves an attacker gaining higher-level privileges within a database system than originally assigned. This can allow the attacker to perform actions or access data beyond their authorized scope, potentially leading to unauthorized control over the database.

**Threat Agents/Attack Vectors:**

- **SQL Injection:** Exploiting vulnerabilities in the application to inject SQL code that grants additional privileges to the attacker.
- **Inadequate Access Controls:** Exploiting weaknesses in the database's access control mechanisms to escalate privileges.
- **Exploiting Configuration Weaknesses:** Leveraging misconfigurations in the database system to gain elevated privileges.
- **Abusing Stored Procedures or Triggers:** Exploiting poorly implemented or vulnerable stored procedures or triggers to escalate privileges.

**Impacts:**

- **Unauthorized Data Access:** The attacker may gain access to sensitive data they were not originally authorized to view.
- **Data Manipulation:** Elevated privileges may allow the attacker to modify or delete data within the database.
- **Complete Database Takeover:** In extreme cases, privilege escalation could lead to full control of the database, enabling the attacker to perform administrative tasks.

**Example Attack Scenarios:**

**SQL Injection for Privilege Escalation:**

- *Attack Vector:* Injecting SQL code to manipulate user privileges.
- *Impact:* Elevating the attacker's access level within the database.

**Exploiting Misconfigured Access Controls:**

- *Attack Vector:* Exploiting misconfigurations to gain higher-level access.
- *Impact:* Escalating privileges beyond the intended scope.

**Abusing Stored Procedures:**

- *Attack Vector:* Exploiting vulnerabilities in stored procedures to gain additional privileges.
- *Impact:* Escalating privileges by abusing the functionality of stored procedures.

**Privilege Escalation through Triggers:**

- *Attack Vector:* Manipulating database triggers to escalate privileges.
- *Impact:* Gaining unauthorized access or control through trigger-based privilege escalation.

Mitigation strategies involve regularly reviewing and updating access control configurations, implementing the principle of least privilege, ensuring secure coding practices to prevent SQL injection, and regularly monitoring and auditing database activity to detect and respond to any suspicious behavior. It's crucial to keep database software and systems updated with the latest security patches to mitigate known vulnerabilities.

- **Risk level:**  Critical
- **State:**  Risk exposed

**Component: User**

Threats of this component

**T-5 Unintended human actions carried out from outside of vehicles**

This threat include:

- Legitimate actors are able to take actions that would unwittingly facilitate a cyber-attack

- **Risk level:**  Critical
- **State:**  Risk exposed

**Component: Web App**

Threats of this component

T-6 Attackers can exploit unsafe XML parsers

General Threat Description:

Unsafe XML parsers can be exploited when processing XML data that contains malicious payloads. Attackers can craft specially designed XML documents to trigger denial of service (DoS) attacks, or even cause arbitrary code execution or data exfiltration by exploiting vulnerabilities in the XML parser.

Threat Agents/Attack Vectors:

- **External Attackers** submitting malicious XML input to the application in an attempt to exploit parser vulnerabilities.
- **Malicious Insiders** using knowledge of the system to craft dangerous XML payloads that bypass security controls.
- **Unvalidated Input** passed to the XML parser from untrusted sources, which can be manipulated to cause unexpected behavior.

Impacts:

- Denial of service (DoS) due to resource exhaustion (e.g., XML billion laughs attack).
- Unauthorized access to sensitive data or execution of arbitrary code.
- System crashes, data corruption, or service disruptions from incorrectly parsed XML data.

Example Attack Scenarios:

1. An attacker submits a malicious XML document containing an entity expansion attack, causing the server to exhaust resources and crash.
2. A crafted XML payload is parsed by an unsafe XML parser, resulting in the execution of arbitrary code on the server.
3. An attacker exploits an unvalidated XML input field, causing the server to parse and leak sensitive information or bypass security checks.

• Risk level: Critical • State: Risk exposed

T-7 Attackers can exploit vulnerable components

General Threat Description:

Vulnerable components, such as outdated libraries, frameworks, or services, can be exploited by attackers to gain unauthorized access or execute malicious actions. These vulnerabilities can allow attackers to compromise the system, steal data, or cause service disruptions.

Threat Agents/Attack Vectors:

- **External Attackers** exploiting known vulnerabilities in third-party libraries or components.
- **Malicious Insiders** taking advantage of unpatched components to escalate privileges or exfiltrate data.
- **Automated Tools** used by attackers to scan for and exploit known vulnerabilities in components.

Impacts:

- Unauthorized access to sensitive data or system controls.
- Potential for remote code execution or privilege escalation.
- Service downtime, data corruption, or loss.

Example Attack Scenarios:

1. An attacker exploits a vulnerability in a third-party library used by the application to execute remote code.
2. A malicious insider exploits an unpatched component to escalate privileges and access sensitive data.
3. An attacker uses an automated vulnerability scanner to identify and exploit outdated software components, gaining unauthorized access to the application.

• Risk level: Critical • State: Risk exposed

T-8 Attackers can intercept or leak sensitive data

General Threat Description:

When sensitive data is transmitted or stored without proper encryption or security controls, attackers can intercept or leak this information, leading to unauthorized access, data theft, or exposure. This can occur due to weak encryption, misconfigurations, or inadequate security measures in place to protect sensitive data.

Threat Agents/Attack Vectors:

- **External Attackers** intercepting unencrypted data during transmission using methods like man-in-the-middle (MITM) attacks.
- **Malicious Insiders** with access to improperly secured data, exploiting their privileges to leak or steal information.
- **Weak Encryption** or outdated cryptographic methods that fail to secure sensitive data during transit or at rest.

Impacts:

- Unauthorized access to sensitive data, including personal, financial, or proprietary information.
- Data breaches, leading to regulatory fines, reputational damage, and loss of customer trust.
- Potential for identity theft, financial fraud, or further exploitation of leaked data.

Example Attack Scenarios:

1. An attacker intercepts sensitive information (e.g., login credentials or personal data) transmitted over an unencrypted HTTP connection.
2. A malicious insider accesses and leaks confidential data stored in plaintext or improperly encrypted in a database.
3. A man-in-the-middle (MITM) attacker exploits weak SSL/TLS configurations, intercepting and decrypting sensitive data being transmitted between a client and server.

• Risk level: Critical • State: Risk exposed

T-9 Attackers can manipulate business workflows

General Threat Description:

Business workflow manipulation occurs when attackers exploit vulnerabilities in a system's process or logic to alter or disrupt normal business operations. By tampering with workflows, attackers can cause unauthorized actions, misroute data, or manipulate processes to benefit malicious goals, leading to business disruption, financial loss, or reputational damage.

Threat Agents/Attack Vectors:

- **External Attackers** exploiting vulnerabilities in workflow logic or APIs to alter business operations.
- **Malicious Insiders** with knowledge of internal workflows who intentionally or accidentally manipulate business processes.
- **Weak Access Control** or improperly secured workflows that allow unauthorized users to perform actions outside their role or authorization.

Impacts:

- Disruption of critical business operations, leading to financial or operational losses.
- Unauthorized transactions or misappropriation of resources.
- Damage to system integrity, data corruption, or data leakage due to workflow manipulation.

Example Attack Scenarios:

1. An attacker manipulates an online order workflow to reroute shipments to an unauthorized address, causing financial losses or theft.
2. A malicious insider alters approval workflows to bypass legitimate checks and approve fraudulent transactions.
3. An attacker exploits a vulnerability in an automated approval system, causing unauthorized changes to sensitive business data or operations.

• Risk level: Critical • State: Risk exposed



**T-10 Attackers can bypass permissions**

**General Threat Description:**

Improperly implemented or enforced server-side permissions allow attackers to bypass access controls and gain unauthorized access to restricted resources.

**Threat Agents/Attack Vectors:**

- **External Attackers** exploiting vulnerabilities in the application's logic.
- **Malicious Insiders** using knowledge to bypass permission checks.
- **Misconfigured RBAC or API endpoints** that fail to enforce proper access checks.

**Impacts:**

- Unauthorized access to sensitive data.
- Potential data breaches, legal consequences, and reputation damage.
- Privilege escalation and unauthorized actions, such as data modification.

**Example Attack Scenarios:**

1. An attacker exploits a flaw to access restricted user data.
2. A user manipulates parameters to escalate privileges and access admin functions.
3. Misconfigured RBAC grants unauthorized users access to sensitive resources.

• **Risk level:**  High • **State:**  Risk exposed

**T-11 Attackers can compromise sessions**

**General Threat Description:**

Session hijacking or fixation occurs when attackers are able to gain unauthorized access to a user's session by stealing or predicting session identifiers. This can happen if session management is insecure, allowing attackers to impersonate legitimate users and access sensitive data or perform unauthorized actions.

**Threat Agents/Attack Vectors:**

- **External Attackers** using techniques like session sniffing, cross-site scripting (XSS), or man-in-the-middle (MITM) attacks to steal session tokens.
- **Malicious Insiders** exploiting weak session management practices to impersonate other users.
- **Weak Session Tokens** or improper session expiration policies that leave sessions vulnerable to hijacking.

**Impacts:**

- Unauthorized access to user accounts, sensitive data, or system resources.
- Potential data breaches, financial fraud, or further exploitation of compromised accounts.
- Loss of trust and reputational damage due to security lapses in session management.

**Example Attack Scenarios:**

1. An attacker intercepts an unencrypted session token in transit using a MITM attack, gaining access to the victim's account.
2. A malicious script injected via XSS steals the session cookie from a user, allowing the attacker to impersonate them and access sensitive data.
3. An attacker guesses or reuses a predictable session ID to take over a valid user session and perform unauthorized actions.

• **Risk level:**  High • **State:**  Risk exposed

**T-12 Attackers can exploit insecure configurations**

**General Threat Description:**

Insecure configurations, such as weak security settings, improper access control, or default settings, can be exploited by attackers to gain unauthorized access or disrupt services. These vulnerabilities often arise from human error, insufficient hardening, or failure to review configurations regularly.

**Threat Agents/Attack Vectors:**

- **External Attackers** exploiting misconfigured security settings or services exposed to the internet.
- **Malicious Insiders** abusing poorly configured systems or excessive permissions.
- **Automated Scanning Tools** that detect default or weak configurations and exploit them for unauthorized access.

**Impacts:**

- Unauthorized access to sensitive resources or data.
- Compromise of system integrity or potential remote code execution.
- Increased attack surface, leading to easier exploitation by attackers.

**Example Attack Scenarios:**

1. An attacker exploits a service left with default credentials or misconfigured access controls, gaining unauthorized access to sensitive data.
2. A misconfigured firewall allows an attacker to bypass security and access a vulnerable internal application.
3. An insecure API configuration exposes critical functions to the public, allowing attackers to exploit them for malicious purposes.

• **Risk level:**  High • **State:**  Risk exposed

**T-13 Attackers can inject malicious commands**

**General Threat Description:**

Command injection occurs when attackers can inject malicious commands into an application, typically through input fields that are not properly sanitized. These commands are executed by the system, allowing attackers to gain unauthorized access, execute arbitrary code, or perform destructive actions on the system.

**Threat Agents/Attack Vectors:**

- **External Attackers** injecting malicious code via input fields, query parameters, or other entry points in the application.
- **Malicious Insiders** exploiting vulnerabilities to execute commands that affect the integrity or availability of the system.
- **Weak Input Validation** or improper handling of user input, allowing attackers to craft commands that the system executes.

**Impacts:**

- Unauthorized execution of commands that compromise system integrity, data, or access.
- Potential for system crashes, data corruption, or unauthorized data retrieval.
- Further exploitation of compromised systems for escalating privileges or launching attacks.

**Example Attack Scenarios:**

1. An attacker injects a command through an unsanitized input field, gaining access to the underlying database and retrieving sensitive data.
2. Malicious input is submitted via an HTTP request that allows the attacker to execute shell commands, altering the server's configuration.
3. An attacker exploits a vulnerable API endpoint to inject commands that cause the application to crash or execute unauthorized actions, disrupting service.

• **Risk level:**  High • **State:**  Risk exposed

T-14 Attackers can inject scripts

General Threat Description:

Script injection, such as Cross-Site Scripting (XSS), occurs when attackers inject malicious scripts into web pages viewed by other users. These scripts are executed in the context of a user's browser, potentially leading to unauthorized access, data theft, or malicious actions on behalf of the victim.

Threat Agents/Attack Vectors:

- **External Attackers** injecting malicious scripts into input fields, URLs, or other entry points that get executed by other users' browsers.
- **Malicious Insiders** exploiting poor input validation or output encoding to inject harmful scripts that compromise user sessions or steal sensitive data.
- **Insecure Web Applications** failing to properly sanitize or escape user inputs, leaving the application vulnerable to script injection.

Impacts:

- Unauthorized access to user data, session hijacking, or credential theft through malicious scripts.
- Data manipulation or destruction by executing harmful scripts on the client-side.
- Loss of user trust and reputational damage if malicious scripts are used to exploit vulnerabilities.

Example Attack Scenarios:

1. An attacker injects a malicious script into a comment form that, when viewed by other users, steals their session cookies and sends them to the attacker.
2. A user is tricked into clicking a link containing a script that injects malicious code into the web page, compromising their account or triggering unauthorized actions.
3. An attacker exploits an input field on an admin panel to inject a script that alters data or performs unauthorized actions within the application.

• Risk level: High      • State: Risk exposed

T-15 Attackers can abuse server-side requests

General Threat Description:

Server-side request forgery (SSRF) occurs when an attacker is able to manipulate a server to make unintended requests to internal or external services, potentially accessing sensitive data, or causing the server to perform malicious actions.

Threat Agents/Attack Vectors:

- **External Attackers** submitting crafted requests that the server forwards to other internal services, allowing access to protected resources.
- **Misconfigured Input Validation** that allows an attacker to control the server's outbound requests, exploiting it to interact with internal systems.
- **Exploiting SSRF in web services** that process requests and interact with databases, internal APIs, or other resources.

Impacts:

- Unauthorized access to internal resources or sensitive data.
- Remote code execution or data exfiltration via internal services.
- Service disruption, denial-of-service (DoS) attacks, or system misconfiguration.

Example Attack Scenarios:

1. An attacker crafts a request that causes the server to query internal resources, exposing sensitive database information.
2. A malicious actor abuses an SSRF vulnerability to interact with a backend service that should only be accessible internally, potentially leading to a data breach.
3. An attacker uses SSRF to trigger unnecessary requests, causing internal services to overload or crash, leading to a denial of service.

• Risk level: Medium      • State: Risk exposed

T-16 Attackers can evade detection without proper logging

General Threat Description:

Without proper logging and monitoring, attackers can evade detection while performing malicious actions on a system. Insufficient or misconfigured logging can prevent timely identification of suspicious activities, making it harder to respond to security incidents or identify compromised systems.

Threat Agents/Attack Vectors:

- **External Attackers** exploiting the absence of detailed logging to perform malicious activities without leaving traces.
- **Malicious Insiders** manipulating or disabling logging to avoid detection while exfiltrating data or modifying critical systems.
- **Misconfiguration** of logging settings that leads to incomplete or missing logs, allowing attackers to act without being recorded.

Impacts:

- Undetected data breaches, system compromises, or malicious actions.
- Delayed or ineffective incident response due to lack of actionable logs.
- Increased risk of lateral movement within the system by attackers, exploiting gaps in logging and monitoring.

Example Attack Scenarios:

1. An attacker compromises an internal server and performs actions (e.g., data exfiltration or privilege escalation) without triggering any alarms because logging is disabled or misconfigured.
2. A crafted serialized object is submitted via an API, causing the system to deserialize the object and execute malicious commands, compromising sensitive data.
3. An attacker exploits an application vulnerability to gain access to critical systems, but the lack of proper logging prevents detection of the breach for an extended period.

• Risk level: Medium      • State: Risk exposed

T-17 Attackers can run malicious code via deserialization

General Threat Description:

Deserialization vulnerabilities occur when an attacker is able to modify or inject malicious code into serialized data, which is then deserialized by the application. This can lead to arbitrary code execution, privilege escalation, or other malicious actions if the deserialization process is not securely handled.

Threat Agents/Attack Vectors:

- **External Attackers** sending specially crafted data to exploit deserialization vulnerabilities in an application.
- **Malicious Insiders** manipulating serialized data for malicious purposes.
- **Weak Input Validation** or improper handling of deserialization routines, allowing attackers to inject harmful payloads into serialized objects.

Impacts:

- Unauthorized execution of malicious code on the system, leading to data breaches, system compromise, or loss of integrity.
- Elevation of privileges or unauthorized access to system resources.
- Disruption of application functionality or service downtime caused by the execution of malicious code.

Example Attack Scenarios:

1. An attacker exploits a deserialization vulnerability to inject a payload that allows them to execute arbitrary code and gain unauthorized access to a server.
2. A crafted serialized object is submitted via an API, causing the system to deserialize the object and execute malicious commands, compromising sensitive data.
3. An attacker manipulates deserialized data in a web application, leading to remote code execution or privilege escalation within the system.

• Risk level: Medium      • State: Risk exposed

Countermeasures to address those threats

Multi-Patient harm view

**Countermeasures Implemented:**

There are no Multi-Patient harm view countermeasures implemented.

Updateability / Patchability view

**Countermeasures Implemented:**

There are no Updateability / Patchability view countermeasures implemented.

Security Use case views

**Countermeasures Implemented:**

• **Component: Database**

There are no Security Use case view countermeasures implemented.

• **Component: User**

There are no Security Use case view countermeasures implemented.

• **Component: Web App**

There are no Security Use case view countermeasures implemented.

Countermeasures validation or testing

Cybersecurity risk controls summary

All security controls identified in the threat model are validated through testing and linked to FDA-required risk evidence.

Countermeasures test results:

|                                                                                                                                                                                                                                                                                                                                                                                     |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div><div><div></div><div>C-4. Adopt the principle of least privilege</div></div><div><div>State: <div><div></div>Recommended</div></div><div>Test status: <div><div></div>Not tested</div></div></div><div>MITRE reference: <a href="#">M1036</a>, <a href="#">M1026</a>, <a href="#">M1018</a> and <a href="#">M1052</a> • Content provided by: <a href="#">MITRE</a></div></div> |
| <div><div><div></div><div>C-16. Block or validate all outbound requests</div></div><div><div>State: <div><div></div>Recommended</div></div><div>Test status: <div><div></div>Not tested</div></div></div><div>MITRE reference: <a href="#">M1031</a> • Content provided by: <a href="#">MITRE</a></div></div>                                                                       |
| <div><div><div></div><div>C-17. Capture and monitor detailed logs for critical actions</div></div><div><div>State: <div><div></div>Recommended</div></div><div>Test status: <div><div></div>Not tested</div></div></div><div>MITRE reference: <a href="#">M1047</a> • Content provided by: <a href="#">MITRE</a></div></div>                                                        |
| <div><div><div></div><div>C-11. Check user roles and permissions on every request</div></div><div><div>State: <div><div></div>Recommended</div></div><div>Test status: <div><div></div>Not tested</div></div></div><div>MITRE reference: <a href="#">M1015</a> • Content provided by: <a href="#">MITRE</a></div></div>                                                             |
| <div><div><div></div><div>C-7. Configure XML parsers to disallow external entities and DTD processing</div></div><div><div>State: <div><div></div>Recommended</div></div><div>Test status: <div><div></div>Not tested</div></div></div><div>MITRE reference: <a href="#">M1055</a> • Content provided by: <a href="#">MITRE</a></div></div>                                         |
| <div><div><div></div><div>C-6. Define security procedures</div></div><div><div>State: <div><div></div>Recommended</div></div><div>Test status: <div><div></div>Not tested</div></div></div><div>MITRE reference: • Content provided by: <a href="#">MITRE</a></div></div>                                                                                                           |
| <div><div><div></div><div>C-5. Define user roles and access privileges</div></div><div><div>State: <div><div></div>Recommended</div></div><div>Test status: <div><div></div>Not tested</div></div></div><div>MITRE reference: • Content provided by: <a href="#">MITRE</a></div></div>                                                                                              |
| <div><div><div></div><div>C-13. Disable unused services, remove default accounts, and apply security updates</div></div><div><div>State: <div><div></div>Recommended</div></div><div>Test status: <div><div></div>Not tested</div></div></div><div>MITRE reference: <a href="#">M1048</a> • Content provided by: <a href="#">MITRE</a></div></div>                                  |
| <div><div><div></div><div>C-12. Enforce MFA, use secure cookies, and invalidate sessions</div></div><div><div>State: <div><div></div>Recommended</div></div><div>Test status: <div><div></div>Not tested</div></div></div><div>MITRE reference: <a href="#">M1056</a> • Content provided by: <a href="#">MITRE</a></div></div>                                                      |
| <div><div><div></div><div>C-15. Ensure proper escaping/encoding of dynamic content and configure CSP</div></div><div><div>State: <div><div></div>Recommended</div></div><div>Test status: <div><div></div>Not tested</div></div></div><div>MITRE reference: <a href="#">M1038</a> • Content provided by: <a href="#">MITRE</a></div></div>                                          |
| <div><div><div></div><div>C-14. Implement prepared statements and validate user inputs</div></div><div><div>State: <div><div></div>Recommended</div></div><div>Test status: <div><div></div>Not tested</div></div></div><div>MITRE reference: <a href="#">M1057</a> • Content provided by: <a href="#">MITRE</a></div></div>                                                        |
| <div><div><div></div><div>C-10. Implement server-side checks and multi-step validation for important transactions</div></div><div><div>State: <div><div></div>Recommended</div></div><div>Test status: <div><div></div>Not tested</div></div></div><div>MITRE reference: <a href="#">M1057</a> • Content provided by: <a href="#">MITRE</a></div></div>                             |
| <div><div><div></div><div>C-2. Implement strong authentication mechanisms</div></div><div><div>State: <div><div></div>Recommended</div></div><div>Test status: <div><div></div>Not tested</div></div></div><div>MITRE reference: <a href="#">M1026</a> and <a href="#">M1032</a> • Content provided by: <a href="#">MITRE</a></div></div>                                           |
| <div><div><div></div><div>C-8. Perform software composition analysis and patch outdated libraries</div></div><div><div>State: <div><div></div>Recommended</div></div><div>Test status: <div><div></div>Not tested</div></div></div><div>MITRE reference: <a href="#">M1054</a> • Content provided by: <a href="#">MITRE</a></div></div>                                             |

C-3. Secure backup storage and access controls

- State: Recommended    Test status: Not tested
- MITRE reference: [M1043](#), [M1035](#) and [M0953](#)    Content provided by: [MITRE](#)

C-1. Use parameterized queries

- State: Recommended    Test status: Not tested
- MITRE reference: [M0818](#) and [M1013](#)    Content provided by: [MITRE](#)

C-18. Use secure data formats and strict whitelisting for deserialization

- State: Recommended    Test status: Not tested
- MITRE reference: [M1038](#)    Content provided by: [MITRE](#)

C-9. Use TLS for communications and protect stored data with encryption

- State: Recommended    Test status: Not tested
- MITRE reference: [M1041](#)    Content provided by: [MITRE](#)

End of Risk controls summary and evidence (for FDA)